K215: Shorter Proofs by Checking Only the Required Interfaces

Current-source audit, direct certificate checking, and normalization of semantic proof obligations

Research contribution for Vladimir Reshetnikov's ProveIt project

22 October 2026 (Pacific time)

Repository snapshot: `cbf94cb552a3b49509e0eee80e7455d4eb5974c6`

Abstract

The final K215 correctness theorem is already a short composition. Further progress should remove repeated obligations below that interface, while keeping the exact 215-state table and the theorem statements fixed. This report audits the current repository rather than relying on its older checkpoint summaries. The maintained Lean endpoint currently imports 174 project modules containing 1,752,650 source bytes; the phase-fifteen summary describes an older 176-module, 2,052,142-byte state. Several attractive proposals have therefore already been adopted.

The principal structural contribution completes an existing cross-system design: a normalized Rocq source-certificate checker whose only lookup requirement is that a returned row has been locally checked. Its soundness does not require balanced-tree cuts to equal subtree sizes, a flattening identity, unique indices, or a separately certified generator. This is the interface already exploited by the maintained Lean checker. The report separates this mathematical simplification from claims about source size or compilation speed. A second, immediately smaller Lean pilot normalizes subtype expressions before transporting witnesses, removing 836 bytes without adding a helper. A more general helper was tested and rejected because it increased total source. The largest remaining operational opportunity is to complete semantic decoder-prefix and residual-runtime contracts for all 6,176 site outcomes. Exact verification scope, reproducible evidence, and remaining integration obligations are recorded with the accompanying code.

Contents

1 Recommendation and scope **3**

2 What is already present at the pinned commit **4**
2.1 Current sources versus checkpoint summaries . . . 4
2.2 Do not propose adopted work as a new result . . . 4
2.3 Where the remaining source volume lies . . . 5

3 A certificate is a checked graph, not necessarily an array **5**
3.1 The existing normalized certificate . . . 5
3.2 The weaker interface . . . 5
3.3 Trees with arbitrary cuts . . . 6
3.4 Relation to the existing Lean proof and earlier proposals . . . 7

4 Performance: keep the claims narrower than the evidence **7**

5 Normalize before abstracting the semantic proof **8**
5.1 The recurring argument . . . 8
5.2 A rejected helper and a smaller replacement . . . 9
5.3 Scope of the method . . . 9

6 The larger remaining opportunity: finish semantic runtime coverage **9**
6.1 What remains active . . . 9
6.2 The correct contract boundary . . . 10
6.3 An incremental integration route . . . 10

7 Integration plan and acceptance criteria **10**

8 Executed checks and deliverable contents **11**
8.1 Lean: proof-body normalization with unchanged public reports . . . 11
8.2 Rocq component: exact normalized certificate and recursive check . . . 12
8.3 Reproduction and repository integration . . . 12

9 Conclusions for this repository **13**

1 Recommendation and scope

Recommendation. Keep the literal K215 machine and the existing endpoint APIs. Simplify the proof by giving each layer the weakest interface that its consumer actually uses. In particular:

1. Treat a source certificate as a finite collection of checked local simulation obligations with a routing operation. The semantic proof needs closure under routing, not equality with a particular serialized list.
2. Normalize subtype operations before moving guarded witnesses. Introduce a new transport abstraction only if enough real clients share it to reduce total maintenance.
3. Finish the existing semantic site-contract interface. Prove the decoder prefix once per source path by a structural theorem, and let the remaining finite checks start at its exact primitive-entry configuration.

The first two items have concrete component evidence in this packet. The Lean patch makes an immediate source reduction. The Rocq interface removes representation-proof obligations, but its first-use source is larger than the present list implementation; it is an optional architecture change whose adoption should be justified by the measured workload. The third item is the main larger integration project. None requires a new 215-state candidate, an altered tape convention, a weaker endpoint, or a proof of consistency as an extra premise.

The exact target. The repository describes K215 as a two-symbol, one-tape machine with 215 working states, blank input, initial state zero, a two-way infinite tape, and a separate halt. Initialization is included. Its dense table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

The maintained endpoints assert, in their respective proof assistants,

$$\mathrm{Halts}(K215, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{Refutable}(\mathrm{ZFC}).$$

Negating both sides gives the desired nonhalting/consistency equivalence in the external formalization. The supplied archive does not change the machine or assert its unconditional nonhalting. See the exact repository endpoint and verification description [2, 1].

An important scope distinction. An external Lean or Rocq theorem about encoded ZFC proofs is not itself an explicitly encoded derivation in ZFC's own proof syntax. The repository states this distinction. Its external theorem and the separately desired internal PA/ZFC derivation should remain separate milestones. The elementary certificate arguments in this report are suitable ingredients for arithmetization, but that observation is not a formal internalization theorem. In particular, one should not simply identify a Lean axiom report with a proof that ZFC derives the endpoint sentence.

What "shorter" measures. There are at least four different quantities: maintained source, checked dependency closure, human mathematical obligations, and checking time. A wrapper can reduce one file while increasing total maintenance. A smaller certificate can make reduction slower. A dependency cut can remove hundreds of kilobytes without changing any proof body. We record these quantities separately and do not add overlapping savings.

2 What is already present at the pinned commit

2.1 Current sources versus checkpoint summaries

The initial inventory used the GitHub connection and a read-only local checkout of commit `cbf94cb552a`. All source claims below refer to that snapshot. The current census follows Lean imports after removing comments and string literals, resolves library roots using `lakefile.toml`, counts each local module once, and excludes Lean's shipped toolchain modules. It is static source analysis, not a new kernel build. The script, source hashes, and complete inventory are included in `evidence/architecture_metrics.json` and `tools/architecture_metrics.py`.

Scope	Project modules	Source bytes
Phase-fifteen historical summary	176	2,052,142
Pinned current K215 endpoint closure	174	1,752,650
Difference	−2	−299, 492

Table 1: The current byte count is freshly measured; the historical byte count is the phase-fifteen report's stated count. This is not a new full-build comparison.

The current closure contains 37,648 physical source lines. The net byte reduction from the historical reported total is about 14.6%. Four old modules leave this closure: `Code254Models`, `KCodeDom`, `KIndisc222`, and `KSeedModel222`. Two enter: `KSeedDirect222` and `KSeedInterface222`. Among common modules, 91 have a different raw source hash; a changed hash does not by itself identify a mathematical change. The historical checkpoint remains evidence about its own exact source/object bindings, not automatic verification of every later source [3].

2.2 Do not propose adopted work as a new result

The current sources already include the following improvements, even where an older proposal index describes them as future work:

- A 1,426-entry normalized correspondence certificate, compared modulo implicit trailing zero bits. The Rocq source theorem checks one direction and obtains the other by symmetry.
- A computed Lean decoder tree: `t215 := ptree p215`. The current stable-program file is 6,867 bytes; the old expanded literal is not the current target.
- Direct compilation of the `C222`/`thm222` calculus, removing the intermediate dominated-calculus step from the active compiler route.
- Direct seed semantics in the current Lean endpoint, and sharing of quotient syntax and quotient-model infrastructure.
- Shared dominated implication, relation-parametric recursion, a smaller PowerSets import boundary, and the stopped-or-synchronized dense-machine proof.
- Subsequent proof-body compression in the current calculus, profile, register, and model files.

These observations come from the maintained source files and recent commits, not merely the historical aggregate table [4, 6, 7, 8, 9].

2.3 Where the remaining source volume lies

The following is a disjoint accounting: first take the logical-calibration roots, then add previously uncounted enumerator/source dependencies, then generic runtime dependencies, then remaining endpoint inputs. It is not the sum of independently measured, overlapping import closures.

Ordered disjoint group	Modules	Bytes
Logical calibration	89	1,110,039
Additional enumerator and source	40	329,522
Additional generic runtime	14	168,755
Remaining machine inputs and endpoint composition	31	144,334
Total	174	1,752,650

Table 2: Current-source decomposition, with root lists and every counted file retained in the census.

The largest individual files include `KSource215Data` (116,343 bytes), `InternalSat` (60,730), `ZF.Zf` (54,075), and `ConstructibleAbs` (50,827). The logical calibration therefore deserves attention as well as certificate data. Yet the final theorem is already small: it composes dense renaming, initialization, binary/source simulation, program equivalence, proof enumeration, and the ZF/ZFC calibration [2, 7]. Golfing that composition has little leverage over its imported proof obligations.

3 A certificate is a checked graph, not necessarily an array

3.1 The existing normalized certificate

A certificate row is a tuple

$$x = (P_1, P_2, j, k),$$

where the finite bit paths P_1, P_2 select corresponding observations in the two register-program trees. The successor labels j and k select the nonzero and zero branches. A local check accepts either two halt observations, or two tests of the same register whose respective successor segments agree [5].

The segment comparison records the finite sequence of incremented registers. Equality of the increment count for every register implies equality of the resulting register file. The segment endpoints are compared using

$$P \simeq_0 Q \quad \Longleftrightarrow \quad \forall i \in \mathbb{N},\ \mathrm{bit}(P, i) = \mathrm{bit}(Q, i),$$

where missing bits are false. This is exactly the implicit-zero convention of the register path semantics. Paths must not be shortened before the consumed prefix has been used to determine increment and break behavior. Normalization concerns the checkpoint relation, not an indiscriminate replacement of operational paths.

The current Rocq checker resolves a successor label through a list lookup. Earlier balanced-tree proposals instead prove that tree lookup equals lookup in the flattened list, certify the sizes of all subtree cuts, and transfer the existing list theorem through this identity [10]. This route is sound. Its representation theorem is nevertheless stronger than the semantic proof needs.

3.2 The weaker interface

Let

$$L : \mathbb{N} \longrightarrow \mathrm{Option}(\mathrm{Path} \times \mathrm{Path})$$

be a routing operation. Write $\mathrm{Local}_L(P, Q, j, k)$ for the current normalized row obligation evaluated with L as successor lookup.

Definition 3.1 (Checked lookup coverage)**.** *A lookup operation has checked coverage if*

$$L(i) = \mathrm{some}(P, Q) \quad \Longrightarrow \quad \exists j, k,\ \mathrm{Local}_L(P, Q, j, k). \tag{LC}$$

The initial segment pair must additionally satisfy its normalized branch obligation for lookup label zero.

The displayed condition can be implemented with a Boolean local checker whose accepted results imply the semantic obligations. It is not an assumption that the certificate generator is correct. The concrete certificate must prove it by computation and a generic coverage lemma.

Theorem 3.2 (Lookup-based normalized source equivalence)**.** *Suppose that lookup coverage (LC) and the initial normalized branch obligation hold for programs p_1, p_2. Then, for every natural-valued register file R,*

$$\mathrm{Halts}(p_1, ([], R)) \quad \longleftrightarrow \quad \mathrm{Halts}(p_2, ([], R)).$$

No injectivity, surjectivity, canonical numbering, sortedness, subtree-size, or list-refinement premise is required of the lookup operation.

Proof structure. For the forward direction use well-founded induction on an actual finite halting time in the source program. At a checked checkpoint, two halt observations immediately transfer halting. Otherwise both programs test the same register. Since their register files agree, they select the same zero or nonzero branch. The source test consumes a positive step. Execute the subsequent finite increment segments using the existing segment lemmas. Equality of increment counts restores equal registers; normalized path equality transfers the source halting witness to the checkpoint selected by L. Lookup coverage supplies the checked local obligation at that selected pair. The remaining source time is strictly smaller, so the induction applies.

For the reverse direction swap the path components of every successful lookup result. The initial branch obligation is symmetric, register equality is symmetric, and the increment-count comparison is symmetric. A returned swapped pair inherits a swapped locally checked row. Thus the same forward theorem applies with the programs exchanged. One does not need to build or verify a second concrete certificate. □

The positive decrement step is essential to this proof. Equality of segment effects alone is not a general license to collapse an infinite sequence of zero-duration source steps. The present theorem uses the inherited finite-run induction at real register tests.

3.3 Trees with arbitrary cuts

Consider a finite routing tree with constructors

$$T ::= \mathrm{Empty} \mid \mathrm{Leaf}(x) \mid \mathrm{Branch}(c, T_l, T_r).$$

The cut c is any natural number. Lookup follows the left child if $i < c$, and otherwise follows the right child with label $i - c$. The supplied Rocq implementation returns a leaf's row only when its residual label is zero; an always-returning leaf, as used in the maintained Lean implementation, would satisfy the same coverage lemma. Define $\mathrm{all}(f, T)$ to test f at every leaf.

Lemma 3.3 (Lookup returns a checked row)**.** *For every f, T, i, x,*

$$\mathrm{lookup}(T, i) = \mathrm{some}(x) \ \wedge\ \mathrm{all}(f, T) = \mathrm{true} \quad \Longrightarrow \quad f(x) = \mathrm{true}.$$

Proof. Induct on T. An empty tree has no successful lookup. A leaf gives the conclusion directly. At a branch, both subtrees satisfy their all-leaf check, and the lookup chooses one of them; apply its induction hypothesis. The proof never inspects the relation between c and either subtree's size. □

Taking f to be the local normalized check establishes (LC). This removes the need for a flattening theorem from the correctness dependency path. It also cleanly distinguishes two questions:

- *Soundness*: if the concrete checker accepts, does the source equivalence follow? Checked coverage answers this without validating cuts.
- *Generator fidelity*: did an external program build the intended tree, preserve every original row in order, and choose efficient cuts? Those are useful diagnostics and completeness/performance properties, but are not additional assumptions of the semantic theorem.

An incorrect cut may route to an unintended row. The branch checker then still compares the actual segment endpoints to the row that was actually returned. It can reject; if it accepts, the successful local simulations are what justify the conclusion. Omitting an unreachable row is likewise harmless only when every successful routing result remains checked and the initial obligation holds. This is a proof about a closed checked relation, not about trusting malformed data.

3.4 Relation to the existing Lean proof and earlier proposals

This is not a claim to have discovered a new general verification principle. The maintained Lean `RMBisim.lookupC_all` already has exactly this shape, and its leaf lookup intentionally does not require an array-style bounds check [6]. The uncompiled guarded-core and sharing-interfaces packets identify the same Rocq opportunity [11]. The present contribution is to complete the interface and instantiate it against the current normalized K215 source certificate, with its concrete checking scope made explicit.

It is also different from merely changing a list to a balanced tree while retaining every representation theorem. The shared semantic proof quantifies over the lookup interface once; the supplied tree instance and same-name source facade preserve the all-register theorem. A future compatibility wrapper can recover the old list-checker API if it is still needed. Otherwise the library can end up maintaining several copies of the same branch and entry soundness argument.

Source-size tradeoff. Counting generic source and facades while excluding common program, RM, and certificate data symmetrically, the earlier checked balanced-refinement stack has 21,768 bytes and 575 lines. The new direct stack has 12,601 bytes and 327 lines, a reduction of 9,167 bytes and 248 lines relative to that alternative. The current production list checker plus its facade has only 9,571 bytes and 224 lines. The direct tree stack therefore adds 3,030 bytes and 103 lines relative to the current list route. This is a simplification of the optional balanced architecture, not an immediate net shortening of the current Rocq source lane. The exact counted file sets and hashes are provided in `source_metrics.json`.

4 Performance: keep the claims narrower than the evidence

The existing normalized certificate has 1,426 entries and induces 2,851 successor lookups in the published lookup census. A prior experiment counted 2,033,059 list-cell visits versus 32,961 tree-node visits, a ratio of about 61.7 [10]. That ratio is an operation-count comparison, not a compilation speedup.

The same prior packet reports a balanced concrete-module compilation time of 15.741 seconds versus 9.238 seconds for its normalized-list baseline. Its recursive kernel check took 313.451 seconds. Tree construction, exact flattening, validity, size, height, reduction strategy, and proof checking all

contribute to the actual cost. The negative result is a reason to change the obligation boundary and then measure, not a reason to promise that any balanced structure will be faster.

The direct checker eliminates the representation-validity and flattening obligations from its semantic theorem. It does not guarantee faster conversion. A sensible experiment holds fixed the programs, normalized row data, toolchain, and checking command, and reports separately:

1. generic proof compilation;
2. certificate-data elaboration;
3. concrete semantic checking;
4. recursive kernel checking;
5. total fresh build time and resulting object sizes.

Generated data should be counted separately from handwritten generic proofs. A repeated test on tiny files or an inherited historical receipt is not a replacement for the exact concrete client.

Adoption decision. Keep the current production list backend as the default until an actual workload justifies switching. The direct tree component is valuable as a smaller alternative to the older balanced-refinement architecture and as a clean interface for future certificate backends. Its current focused measurements do not justify describing it as a faster or smaller replacement for the present production list path. The normalization-first Lean patch has the clearest immediate source-shortening benefit.

5 Normalize before abstracting the semantic proof

5.1 The recurring argument

Let D be an ambient domain and let $P : D \to \mathrm{Prop}$ define a subtype

$$D_P = \{x : D \mid P(x)\}.$$

A guarded quantifier often restricts all relevant values to P already. For example, if $y \in x$ and $P(x)$, transitivity may imply $P(y)$. In a graph-membership argument, closure of a pair code may force both coordinates into the subtype. The current constructibility development repeatedly constructs subtype witnesses, proves these membership facts, and rewrites the same ambient/subtype relation [12].

The mathematical pattern is a theorem about supported guards, not a stronger global model hypothesis. It is useful to understand this pattern even when adding a new library abstraction is not the best implementation.

Theorem 5.1 (Guarded existential restriction)**.** *Let $G, F : D \to \mathrm{Prop}$ and assume*

$$\forall x,\ G(x) \to P(x).$$

Then

$$(\exists x : D,\ G(x) \wedge F(x)) \quad \longleftrightarrow \quad (\exists x : D_P,\ G(x.1) \wedge F(x.1)).$$

Proof. In the forward direction send a witness x to the subtype witness $\langle x, h_G(x)\rangle$. In the reverse direction forget the subtype membership proof. The same guard and body are retained. □

Theorem 5.2 (Guarded universal restriction)**.** *Under the same support hypothesis,*

$$(\forall x : D,\ G(x) \to F(x)) \quad \longleftrightarrow \quad (\forall x : D_P,\ G(x.1) \to F(x.1)).$$

Proof. Restriction is immediate. For extension, use the guard to construct the required subtype witness and apply the restricted statement to it. □

Product-valued witnesses give the corresponding multi-coordinate statement. In implementation it can be more readable to export two- and four-coordinate corollaries with named binders than to make every client manipulate a product. However, helpers should be introduced only where their aggregate use pays for their own definitions.

5.2 A rejected helper and a smaller replacement

An initial pilot introduced a reusable guarded-subtype helper. Its generic lemmas compiled without axioms, and individual witness blocks became shorter, but the helper cost 1,176 bytes. Counting the helper and the changed clients, the pilot grew by 904 bytes. It is not recommended as a proof-shortening patch. This negative result is retained in the evidence so that the recommendation does not mistake a smaller client for smaller total maintenance.

The successful replacement instead uses the existing subtype evaluation equalities first. Normalize pair codes, natural-number codes, and bit codes to their ambient values; then move the already-supported witnesses. This removes repeated conversions at each local use without introducing a new module.

The resulting five-theorem patch changes the proof bodies of `functional_iff`, `isFunctionOn_iff`, `isEnvOn_iff`, `atom_clause`, and `bin_clause`. The complete `ConstructibleAbs.lean` file shrinks from 50,827 to 49,991 bytes and from 917 to 911 lines: 836 bytes and six lines removed, with no new imports, definitions, helper declarations, or changed theorem statements. The byte reduction is modest (about 1.65%), but it makes the witness argument more direct and has a small review surface. Its exact checks are described in Section 8.

5.3 Scope of the method

This does not justify replacing the entire internal-formula-code argument by an ordinary structural induction over external syntax. The model development handles represented formula codes, including codes that may be nonstandard from the external point of view. Its represented-rank descent has a different job from the elementary witness transport. The pilot is deliberately limited to the latter.

Nor should a small source-line reduction automatically be called an improvement. A long generic theorem that saves two lines at one client can make the proof harder to read. The pilot's useful lesson is to normalize the concrete semantic expressions first and only then assess whether a shared theorem is still needed.

6 The larger remaining opportunity: finish semantic runtime coverage

6.1 What remains active

The current `KSim.Covered` interface still asks for the Boolean `checkEdge` result at every consumed source leaf [13]. The site inventory contains exactly

$$4,751 \text{ leaves}, \qquad 6,176 \text{ branch outcomes}.$$

The instruction counts are 1,900 increments, 1,425 decrement tests, 1,327 continues, 98 breaks, and one halt. A decrement contributes two outcomes; every other leaf contributes one. The maintained

Lean coverage is divided into nine batches containing 105 kernel-check blocks. These are source/data counts from the current tree, not newly timed compilation results.

The repository already contains generic semantic-site interfaces, an exact decoder-prefix theorem, and partial composition pilots [14, 15]. A proof for one leaf, or two outcomes of a decrement, does not establish the full 6,176-outcome replacement. The next substantial work is to close that finite but delicate integration gap.

6.2 The correct contract boundary

Use a semantic edge specification: for every valid represented register file and padding parameter, the machine executes the intended register instruction and reaches the exact successor dispatch representation; a halt leaf reaches a genuine terminal configuration. The present `KSim` halting-transfer argument additionally needs positive progress on simulated source steps. A different simulation discipline could allow zero-step transitions together with a separate well-founded rank; no such replacement is asserted here.

Split the proof at the exact primitive-entry port already supplied by the decoder-prefix theorem:

$$\mathrm{Dispatch}(Q) \longrightarrow^{*} \mathrm{Port}(\mathrm{reverse}(\mathrm{pre}), b, q) \longrightarrow^{+} \mathrm{SuccessorDispatch}(Q', R').$$

The prefix theorem keeps the complete consumed source word on the left of the head. The residual contract must consume this exact port, including the marker, path tail, gap expression, physical register order, and branch assumption. Merely identifying a physical state or a short suffix does not identify the full continuation when decoder roles share states.

6.3 An incremental integration route

First keep `checkEdge` as a producer of semantic contracts. This preserves a working fallback while changing `KSim` to consume the semantic statement. Then add a second producer using the structural decoder prefix and a residual symbolic check. Mixed coverage lets different leaves use different producers under the same endpoint theorem.

For the current source, generate a total classification of all 4,751 leaves, including both decrement outcomes. Prove that every classified leaf has one of the accepted producers. The finite coverage object should fail if any leaf is missing, if a branch polarity is swapped, or if the halt case is classified as an ordinary return. Only after that theorem is checked should the redundant full-prefix runs be retired.

Next identify truly shared residual contracts. A frame lemma may factor an untouched continuation, but it must state precisely where the head can move and where the marker is found. An unrestricted "arbitrary left context" principle is false for code that scans into that context. The earlier left-frame proposal is a useful starting point, not already a theorem about every K215 residual [16].

This architecture can reduce repeated decoder execution in the proof while leaving the physical transition table intact. It also offers a cleaner proof surface for related 215-state candidates, since a changed decoder can reuse the same semantic primitive contracts when its exact entry hypotheses are re-established.

7 Integration plan and acceptance criteria

Preserve the public source name `Source215.p215_dagPerm`. Likewise retain `blank_halts_iff_zfc_refutation` in `KEndpoint215`. Keep exact machine hashes and public theorem types in the integration checks. For changed producer-generated sources, change the producer as well and compare regenerated bytes. Re-run affected descendants; source identity and inherited compiler receipts should not be conflated.

Stage	Change	Required evidence
1. Lean normalization	Normalize subtype operations before transporting witnesses.	Unchanged theorem types; fresh module and client checks; no assumption growth.
2. Optional source interface	Generalize normalized checking over checked lookup; add the tree producer if useful.	Exact all-register source theorem; printed assumptions; concrete certificate check.
3. Production adoption	Rebuild affected descendants after selecting either component.	Current-toolchain checks, including sparse/dense endpoints; source and theorem identity.
4. Site interface	Let simulation consume semantic edge contracts with the old checker as a producer.	Existing endpoint clients still check.
5. Complete residuals	Structural decoder prefix plus exact residual contracts for every outcome.	Total 4,751-leaf/6,176-outcome coverage, bootstrap and terminal cases preserved.
6. Cleanup	Remove displaced duplicated code and synchronize maintained generators.	Before/after maintenance counts and reproducible regeneration.

Table 3: A sequence of small, separately reviewable changes. Stages with completed component evidence are identified in the following section; a component receipt is not a claim that all stages have passed.

The criterion for retaining an abstraction is a smaller total burden after counting generic helpers, adapters, and synchronized generators. For timing, compare complete relevant workloads under the same toolchain. For logical trust, inspect the new theorem's assumptions and the exact endpoint's assumptions independently.

8 Executed checks and deliverable contents

8.1 Lean: proof-body normalization with unchanged public reports

The accepted patch was replayed with Lean 4.32.0 in a fresh output directory. The 25 successful compiler invocations comprise 21 unchanged dependency modules, the original `ConstructibleAbs` module, an API-reporting client, the modified module, and the identical API-reporting client. Both complete module compilations include the unchanged downstream proofs inside that module.

The six public type and assumption reports—the five changed theorems and the downstream `vEqL`—are byte-for-byte identical between the two builds. Each lists precisely `propext`, `Classical.choice`, and `Quot.sound`. Both target logs and both API-client logs have zero warnings and errors. The patch passes `git apply -check`. An independent source review additionally verified that masking only the five proof bodies leaves the complete original and proposed files identical, including section directives. The fresh-build evidence and the source review are separate records, not two independent compilations.

The paired target compilations took 0.938 seconds for the original and 1.009 seconds for the proposal in this one run. No speed improvement is established. The supported result is a checked reduction of 836 source bytes, with unchanged declarations and printed assumptions. Other descendants and the complete K215 endpoint were not rebuilt.

8.2 Rocq component: exact normalized certificate and recursive check

The delivered source-equivalence component was built from fresh sources with Coq 8.18.0, compiled with OCaml 4.14.1. All 18 module invocations succeeded. They include the inherited register semantics and programs, the current 1,426-row normalized certificate, the production list baseline, the generic checked-lookup theorem, the direct tree instance, a same-name `K215F.Source215` facade, a typed client, and eight boundary controls. The concrete theorem retains the quantifier over *every* natural-valued register file:

$$\forall R, \quad \texttt{rm_halts}\ p215\ ([], R) \longleftrightarrow \texttt{rm_halts}\ dagPerm\ ([], R).$$

The delivered client and concrete source theorems print as closed under the global context. A recursive `coqchk` run on `SourceClient` also succeeded, taking 259.463 seconds in this run. The evidence records the exact commands, source and log hashes, elapsed times, and process peak resident memory.

Concrete source-check module	Seconds	Peak RSS (KiB)
Current list baseline	8.968	456,896
Direct checked-tree instance	11.135	527,844

Table 4: One serial fresh run using the same compiler and inherited programs. These are module invocations, not full endpoint build times or statistically stable benchmarks.

The direct instance is about 24% slower than the production list source module in this observation. Its separate facade and typed client also take time; the table is not a claim about the entire build. A packed-payload tree variant was tried and rejected because it had a larger adapter and slower focused checks. The selected implementation uses expanded rows. Both the source-size comparison and this negative timing result support retaining the current list backend as the default.

The production repository uses Rocq 9.1.0. Its inherited `RMEquiv.v` requires a small explicit compatibility import under Coq 8.18; the packet retains the original 9.1 source and hashes both the delivered and compiled 8.18 variants. The build driver leaves that inherited source unchanged on its 9.1 branch, but that branch has not been executed here. This receipt establishes a concrete Coq 8.18 source-layer theorem and recursive check. It is not a Rocq 9.1 production acceptance receipt, nor a new binary-machine/ZFC endpoint build.

8.3 Reproduction and repository integration

The archive is intended to be unpacked as a research contribution, for example below the existing K215 simplification-proposals directory. Its root `README.md` gives the current adoption recommendation, file map, and commands. The original checkout is unchanged.

```
python3 verify_packet.py --repo /path/to/ProveIt
python3 lean-normalization/reproduce.py \
  --repo /path/to/ProveIt --lean /path/to/lean \
  --out /new/lean-output
python3 rocq/build.py --out /new/rocq-output --baseline --recursive
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The packet verifier checks hashes and recorded results; it does not run a proof kernel. The two replay drivers perform the fresh component checks. The source census can also be rerun against a matching checkout with the supplied analysis script. No compiler binaries, build objects, or precompiled proof caches are needed from this archive.

Artifact	Purpose
`report.tex`, `results.tex`, `report.pdf`	Editable report, executed-check section, and rendered document.
`lean-normalization/`	Reviewable patch, full proposed source, source manifest, fresh replay driver, API audit, logs, and failed-edit history.
`rocq/`	Generic and concrete proofs, source facade, typed client, boundary cases, inherited dependencies and licenses, replay driver, metrics, and final recursive-check evidence.
`experiments/rejected-helper/`	The rejected Lean helper and client, with whole-change accounting.
`evidence/`, `tools/`	Current-source census, snapshot provenance, independent component reviews, and census script.
`MANIFEST.json`, `verify_packet.py`	Exact file hashes and evidence-binding checks.

Table 5: Deliverables and their verification roles.

9 Conclusions for this repository

The shortest route to another useful improvement is to exploit the interfaces that the existing proofs already reveal. The normalized source proof only needs closure under checked lookup. The subtype proofs often only need a guarded witness restriction lemma. The runtime simulation only needs semantic edge behavior, even though its current producer performs a full bounded symbolic execution at every site.

The accompanying components make these distinctions concrete. They preserve the accepted K215 target and distinguish checked source-layer results from a future full production endpoint rebuild. The larger decoder/residual project remains worthwhile precisely because the smaller preliminary interfaces are already available; it should be pursued as a complete coverage theorem with exact ports and positive progress, rather than another isolated successful decoder example.

References

[1] ProveIt contributors. Formal correctness of the exact 215-state machine. Pinned repository verification description.

[2] ProveIt contributors. `KEndpoint215.lean`. Exact Lean endpoint.

[3] ProveIt contributors. Fifteenth K215 proof-simplification checkpoint. Historical evidence and scope.

[4] ProveIt contributors. `Source215.v`. Current normalized source equivalence.

[5] ProveIt contributors. `NormalizedBisim.v`. Current Rocq normalized checker and finite-run soundness argument.

[6] ProveIt contributors. `RMBisim.lean`, and `RMNormalizedBisim.lean`. Semantic bisimulation, checked lookup, and normalized relation.

[7] ProveIt contributors. `KEndpoint.lean`. Shared current endpoint and direct-seed calibration route.

[8] ProveIt contributors. `KSource215Program.lean`. Computed decoder tree.

[9] ProveIt contributors. Proof-simplification proposal inventory. Historical packet descriptions; individual adoption status must be checked against current source.

[10] ProveIt research packet. Composed normalized and balanced source certificate. Prior exact-flattening route, operation counts, and measured negative timing result.

[11] ProveIt research packets. Guarded-core refinement review; Sharing semantic interfaces. Earlier checked-lookup designs.

[12] ProveIt contributors. `ConstructibleAbs.lean`. Constructibility, represented codes, and subtype witness arguments.

[13] ProveIt contributors. `KSim.lean`. Current Boolean coverage and register-machine simulation.

[14] ProveIt research packets. Checked semantic proof contracts; Semantic coverage. Earlier component proofs and their limited integration scope.

[15] ProveIt contributors. `KDecoderPrefix215.lean`. Exact zipper entry retaining the entire consumed source word.

[16] ProveIt research packet. Further proof simplification, round 13. Proposed guarded left-frame lifting and other explicitly scoped work.